

Atv Extra: Analise o possível problema e apresente uma solução (17:20)

```
#include <mpi.h>
#include <stdio.h>
#include <stdlib.h>

void initVetor(int *valor_enviado, int tamanho, int rank) {
    for (int i = 0; i < tamanho; i++) {
        valor_enviado[i] = rank + i + 1;
    }
}

int main(int argc, char *argv[]) {
    MPI_Init(&argc, &argv);

    int rank, size;
    int tamanho;

    MPI_Comm_rank(MPI_COMM_WORLD, &rank);
    MPI_Comm_size(MPI_COMM_WORLD, &size);

    if (argc < 2) {
        printf("Uso: %s tamanho\n", argv[0]);
        return 1;
    }

    tamanho = atoi(argv[1]);

    int valor_enviado[tamanho], valor_recebido[tamanho];
```

```
    if (size != 2) {
        if (rank == 0)
            printf("Execute com exatamente 2 processos.\n");

        MPI_Finalize();
        return 0;
    }

    initVetor(valor_enviado, tamanho, rank);

    if (rank == 0) {

        MPI_Send(&valor_enviado, tamanho, MPI_INT, 1, 0, MPI_COMM_WORLD);

        MPI_Recv(&valor_recebido, tamanho, MPI_INT, 1, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    } else if (rank == 1) {

        MPI_Send(&valor_enviado, tamanho, MPI_INT, 0, 0, MPI_COMM_WORLD);

        MPI_Recv(&valor_recebido, tamanho, MPI_INT, 0, 0,
                 MPI_COMM_WORLD, MPI_STATUS_IGNORE);

    }

    printf("Processo %d recebeu os dado\n", rank);

    MPI_Finalize();
    return 0;
}
```




IMD1116

COMPUTAÇÃO DE

ALTO DESEMPENHO

MPI:

Prof. Samuel Xavier de Souza
Prof. Carla Santana

PROGRAMAÇÃO PARALELA

- Objetivos deste tópico

Revisão Deadlock (Impasse)



Carros ficaram travados no cruzamento entre as avenidas Amazonas e Contorno. — Foto: Globocop

Recursos

- Um recurso é algo que pode ser adquirido, usado e liberado com o passar do tempo.
- Recursos Preemptíveis X Não preemptíveis

Recursos

- Um recurso preemptivo é aquele que ***pode ser retirado do processo*** proprietário sem nenhum prejuízo. (Memória Ram)
- Um recurso não preemptivo é aquele que não pode ser retirado do processo proprietário sem nenhum prejuízo. (Impressão)

Impasse

“Um conjunto de processos estará em situação de impasse se todo processo pertencente ao conjunto estiver esperando por um evento que somente outro processo desse mesmo conjunto poderá fazer acontecer.”

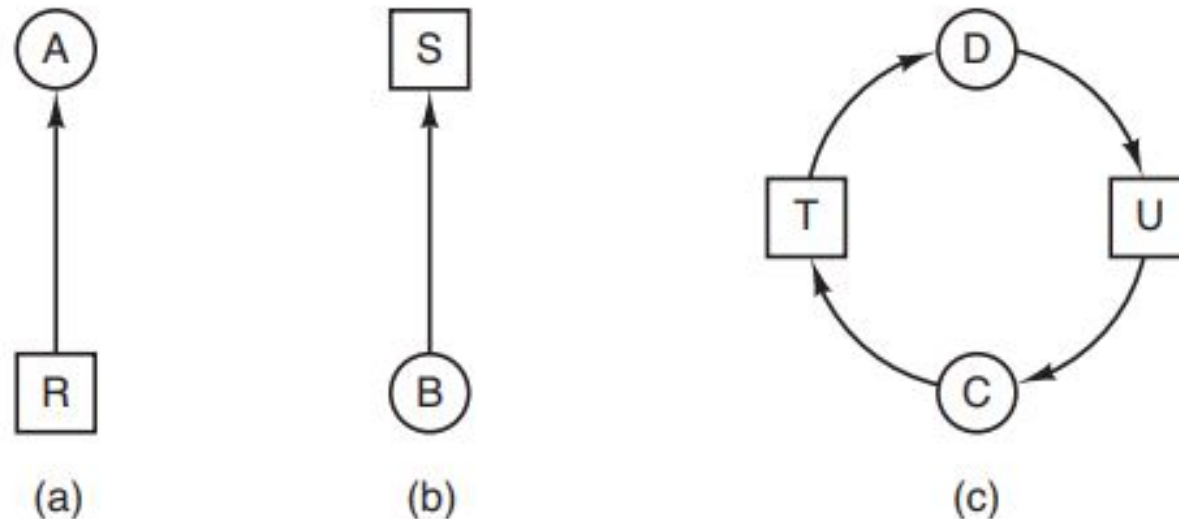


Figura 6.3 Grafos de alocação de recursos. (a) Processo de posse de um recurso. (b) Processo requisitando um recurso. (c) Impasse.

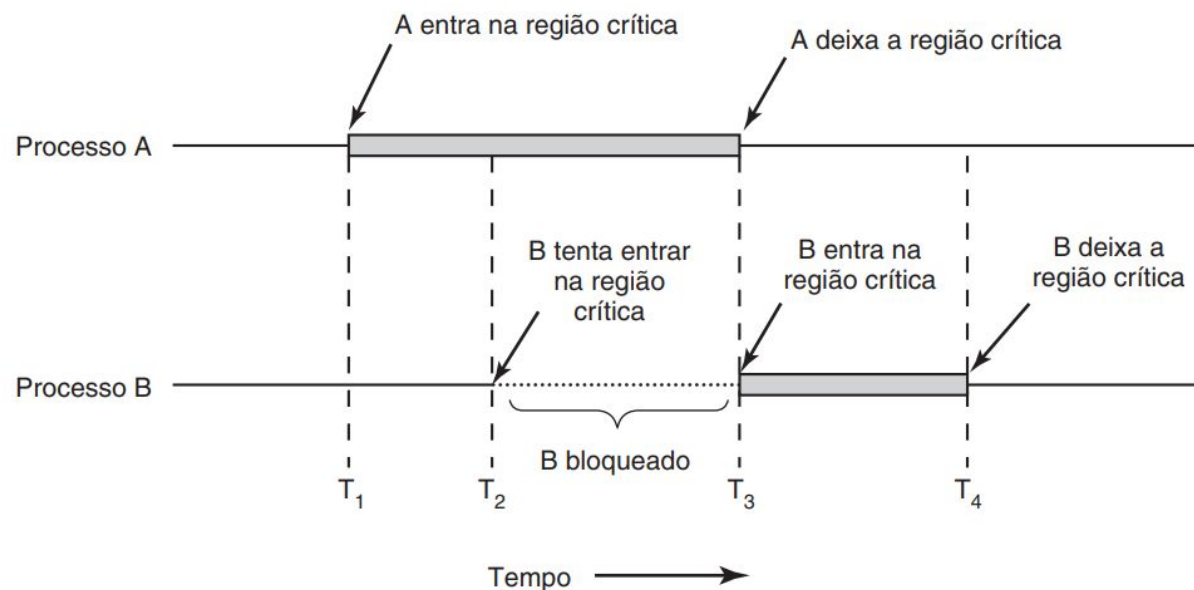
Condição para ocorrência de Deadlock

1. Condição de exclusão mútua
2. Condição de posse e espera
3. Condição de não preempção
4. Condição de espera circular

Condição para ocorrência de Impasses de Recursos

1. Condição de exclusão mútua

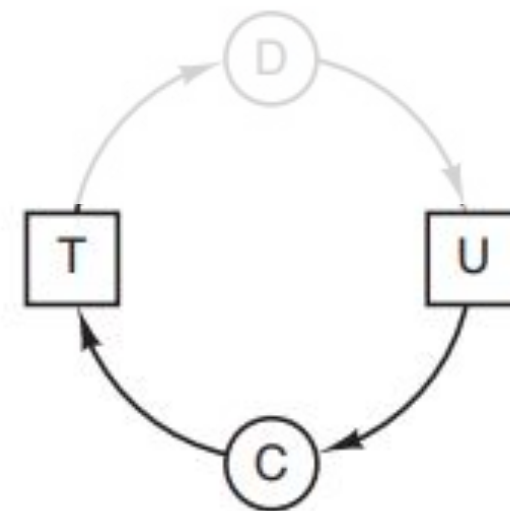
- “Em um determinado instante, cada recurso está em uma de duas situações: ou associado a um único processo ou disponível.”



Condição para ocorrência de Impasses de Recursos

2. Condição de posse e espera

“Processos que, em um determinado instante, retêm recursos concedidos anteriormente podem requisitar novos recursos.”



Condição para ocorrência de Impasses de Recursos

3. Condição de não preempção

“Recursos concedidos previamente a um processo não podem ser forçosamente tomados desse processo — eles devem ser explicitamente liberados pelo processo que os retém.”

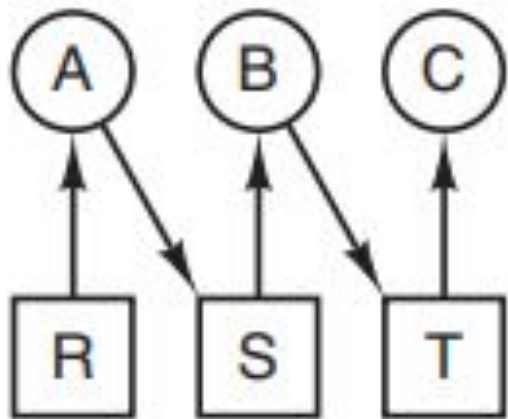
Condição para ocorrência de Impasses de Recursos

4. Condição de espera circular

“Deve existir um encadeamento circular de dois ou mais processos; cada um deles encontra-se à espera de um recurso que está sendo usado pelo membro seguinte dessa cadeia.”

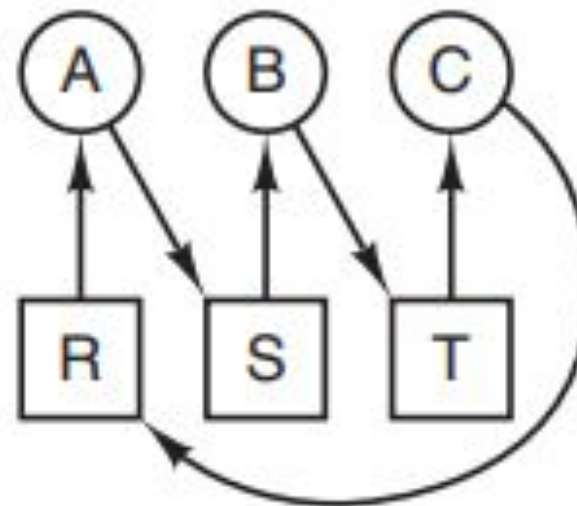
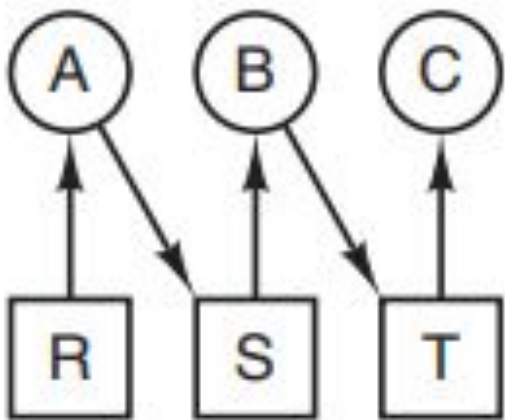
Condição para ocorrência de Impasses de Recursos

4. Condição de espera circular



Condição para ocorrência de Impasses de Recursos

4. Condição de espera circular



Situação de
Impasse

Evitando Deadlock programando com MPI

- Ocorrência de ciclos de dependência entre os processos durante a troca de mensagens entre eles.
- Exemplo:
 - Ao enviarmos uma mensagem grande do processo 0 para o processo 1, verificamos que o espaço de armazenamento no destino é insuficiente.
 - A rotina de envio deve esperar até que o usuário providencie espaço de memória suficiente (através da chamada de uma rotina de recepção).

Processo 0	Processo 1
MPI_Send(1)	MPI_Send(0)
MPI_Recv(1)	MPI_Recv(0)

Evitando Deadlock programando com MPI

1. Ordenar as operações de envio adequadamente

Processo 0	Processo 1
MPI_Send(1)	MPI_Send(0)
MPI_Recv(1)	MPI_Recv(0)

Processo 0	Processo 1
MPI_Send(1)	MPI_Recv(0)
MPI_Recv(1)	MPI_Send(0)

Evitando Deadlock programando com MPI

- 2 Fornecer explicitamente um bufer para envio da mensagem

Processo 0	Processo 1
MPI_Send(1)	MPI_Send(0)
MPI_Recv(1)	MPI_Recv(0)

Processo 0	Processo 1
MPI_Bsend(1)	MPI_Bsend(0)
MPI_Recv(1)	MPI_Recv(0)

Evitando Deadlock programando com MPI

- 3 Utilizar operações de envio e recepção não bloqueantes

Processo 0	Processo 1
MPI_Send(1)	MPI_Send(0)
MPI_Recv(1)	MPI_Recv(0)

Processo 0	Processo 1
MPI_Isend(1)	MPI_Isend(0)
MPI_Irecv(1)	MPI_Irecv(0)
MPI_Waitall	MPI_Waitall

Evitando Deadlock programando com MPI

4 MPI_Sendrecv

Processo 0	Processo 1
MPI_Send(1)	MPI_Send(0)
MPI_Recv(1)	MPI_Recv(0)

Processo 0	Processo 1
MPI_Sendrecv(1)	MPI_Sendrecv(0)

MPI_Sendrecv

```
int MPI_Sendrecv(  
    const void *vet_envia,  int cont_envia, MPI_Datatype tipo_envia,  int dest,    int tag_envia,  
    void *vet_recebe, int cont_recebe, MPI_Datatype tipo_recebe, int origem, int tag_recebe,  
    MPI_Comm com, MPI_Status *estado)
```

“Podemos pensar nessa rotina como sendo um MPI_Isend, seguido de um MPI_Irecv, e um par de MPI_Waits. Assim, efetivamente, o envio e a recepção da mensagem acontecem em paralelo”

PROGRAMAÇÃO EM MEMÓRIA DISTRIBUÍDA

Tarefa 16:

Desenvolver **um escalonador dinâmico de tarefas utilizando MPI no modelo Líder–Trabalhador** (Master–Worker).

O processo líder será responsável por distribuir tarefas aos trabalhadores de forma dinâmica, **enviando novas tarefas conforme os trabalhadores finalizam as anteriores** .

A aplicação do escalonador será a paralelização do cálculo de números primos em um intervalo de valores.

Referências

